

SocioMart Functional Document

1. Project Overview

My First Spring API is a full-stack web application that connects home chefs in a residential society with neighbours who want fresh, homemade food. It removes the need for WhatsApp group chaos by giving every kitchen a professional online menu, order management dashboard, and earnings tracker.

1.1 The Problem It Solves

- Home chefs currently take orders through scattered WhatsApp messages — orders get missed, quantities sell out silently.
- Buyers have no single place to see which kitchens are open today and what is still available right now.
- Sellers have no record of daily revenue, pending payments, or which dish is most popular.

1.2 The Solution in One Line

One shared platform: sellers publish daily offerings in under a minute (manual form, saved Favourites, or paste a WhatsApp message and let the app parse it), buyers order with 3 taps, and both sides get live status tracking with automatic inventory.

1.3 Who Uses It (3 Roles)

Role	What They Do	How They Log In
Buyer	Browse kitchens, add items to a cart, place orders (prepaid or cash-on-delivery), track status	Mobile number + OTP
Seller	Create offerings, control inventory, mark sold-out, manage order pipeline, view earnings	Mobile number + OTP (seller-flagged account)
Admin	Approve/verify new kitchens, monitor the whole platform	Admin console

2. Buyer Experience

2.1 Home — Live Kitchen Discovery

The buyer lands on a card grid of every kitchen in the society. Each card shows the kitchen's name, food description, rating and a live "available today" badge. A search box filters by name or dish instantly. Clicking a card fires a MENU_VIEW analytics event, which powers the seller dashboard's real "views today" metric.

2.2 Kitchen Menu & Ordering

Inside a kitchen the buyer sees today's offerings with live remaining quantity ("only 4 of 10 left"). Items that are sold out or past their order cutoff are visibly disabled — the server enforces the same rule again at order

time, so the UI can never lie its way into a bad order.

2.3 The Ordering Flow (step by step)

Step	What Happens	Server Guarantee
1. Add to cart	Buyer picks quantity per item	Quantity capped at remaining stock
2. Draft order	POST /api/buyer/orders/draft prices the cart	Server recomputes price from DB — client cannot tamper
3. Buyer details	Name, flat number, mobile confirmed	Validated and stored on the order
4. Place order	POST /api/buyer/orders/place	Inventory decremented atomically; sold-out rejected with HTTP 409
5. Track	Order shows PENDING -> CONFIRMED -> DELIVERED	Status transitions validated by the seller pipeline

3. Seller Experience

3.1 Live Dashboard — The Command Centre

The seller dashboard answers four questions at a glance: How many people viewed my menu today? (real analytics count), What's still in stock?, How much have I earned? (split into confirmed-today, pending and this-month), and What orders need action?

3.2 Three Ways to Publish an Offering

Method	How It Works	Safety Rails
Manual form	Name, price, unit, max quantity, cutoff time	HH:mm cutoff validation, positive price/quantity checks
Favourites (templates)	Save a dish once; republish any day in one click	Strict 3-template cap — 4th save returns HTTP 409 "Maximum 3 favourites"
WhatsApp Quick Post	Paste a real WhatsApp message; the parser extracts name, price, quantity and ready-by time	Parser only suggests — nothing is published without the seller confirming

3.3 Inventory Control — Live Steppers

Each live offering has +/- quantity steppers and a one-tap Sold Out button:

- Every click is a PATCH /api/seller-app/products/{id}/inventory with a delta — server-side arithmetic, race-safe.
- Going below zero is rejected (HTTP 400) — stock can never go negative.

- Unlimited-quantity items have no stepper ("no quantity limit" message) but can still be force-marked sold out.
- Cross-seller protection: seller B calling the endpoint on seller A's item gets HTTP 403.

3.4 Order Pipeline & Earnings

Orders move through PENDING -> CONFIRMED -> DELIVERED (or CANCELLED). The Daily Summary aggregates plates and revenue per dish; clicking a dish drills into Item Details showing each buyer row. A cancelled order is excluded from plates, revenue and earnings — counts stay consistent across dashboard, summary and drill-down.

4. Admin Console

The admin console lists all kitchens with their verification status. Admins can approve new sellers before they appear publicly, keeping the marketplace trustworthy. It is a separate single-page view hitting the `/api/admin/*` endpoints.

4.1 Admin Features

- Dashboard with total orders, paid/will-pay-later breakdown, buyer/seller counts
- Buyer management (list all buyers)
- Seller management (list all sellers, pending approvals)
- Kitchen management (list all kitchens, approve/reject/suspend)
- Offerings management (browse all product listings)
- Order monitoring (full order list with status and payment)
- Enquiries management (customer inquiries)
- Analytics (platform metrics)

5. Key Business Rules & Validations

Every rule below is enforced on the server, not just in the browser — a hostile or buggy client cannot bypass any of them.

Rule	Enforcement	Verified By
Order cutoff time	Strict 24-hour HH:mm; invalid (e.g. 25:99) rejected HTTP 400; ordering after cutoff rejected	E2E I-1b/c
Stock can't go negative	Delta pushing stock below 0 -> HTTP 400	E2E D3
Sold-out unorderable	Draft & place reject sold-out items (HTTP 409)	probe
Template cap = 3	4th save -> HTTP 409 "Maximum 3 favourites"	E2E F2
Template independence	Publishing creates a new Product — live edits never touch the saved favourite	E2E G1
Seller ownership	Every mutation checks kitchen owner; else HTTP 403	E2E E1
Unlimited quantity	No stepper; delta -> "no quantity limit"; sold-out still allowed	E2E G2/G3
Quick Post never auto-publishes	Parse is read-only — offerings unchanged after parse	E2E H2
Cancelled != revenue	Excluded from plates, revenue, earnings, drill-down	E2E L1-L7
Earnings states	pending = PENDING only; confirmedToday = CONFIRMED/DELIVERED	E2E K1/K2
Server-side pricing	Totals recomputed from DB — client totals ignored	E2E I1
Duplicate order prevention	Same buyer+kitchen+details within window rejected HTTP 400	Live test
Shared DB across roles	Seller and Admin see identical order data	Live test

6. Payment Status Flow

The `PaymentStatus` enum supports three states: `PENDING`, `PAID`, `WILL_PAY_LATER`.

`OrderService.consumeOrder` maps payment status to order status:

- `PENDING` -> `CONFIRMED` (legacy path)
- `PAID` -> `CONFIRMED`
- `WILL_PAY_LATER` -> `ORDERED`

`OrderService.updatePaymentStatus()` sets `CONFIRMED` when `PAID` && `ORDERED`.

7. Demo Data

The `DemoDataSeeder` seeds 15 kitchens with image/Instagram support on first boot. Aarti Kitchen pre-order demo features Puran Poli with `isPreorder=true`, `preorderType=FIXED`, future Monday date, cutoff 12:00, qty 30, price 70. Orders are seeded with a `PAID/WILL_PAY_LATER` mix. New `seedEnquiriesIfEmpty()` and `seedFavouritesIfEmpty()` methods populate those tables.

Note: H2 in-memory database means custom data disappears on restart; demo data re-seeds on every boot (accepted limitation).